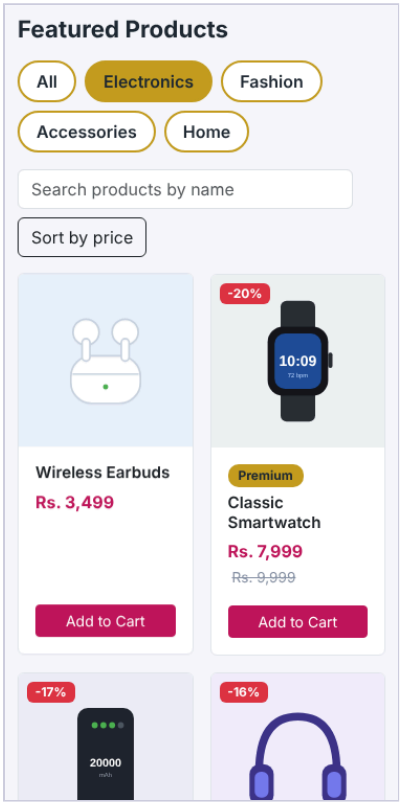
Muhammad Abdul Rehman Azhar

Registration number: FA24-BCS-058
CSC336 Web Technologies · FA26 BCS A Lab Mid Term · Instructor: Muhammad Usman Akram

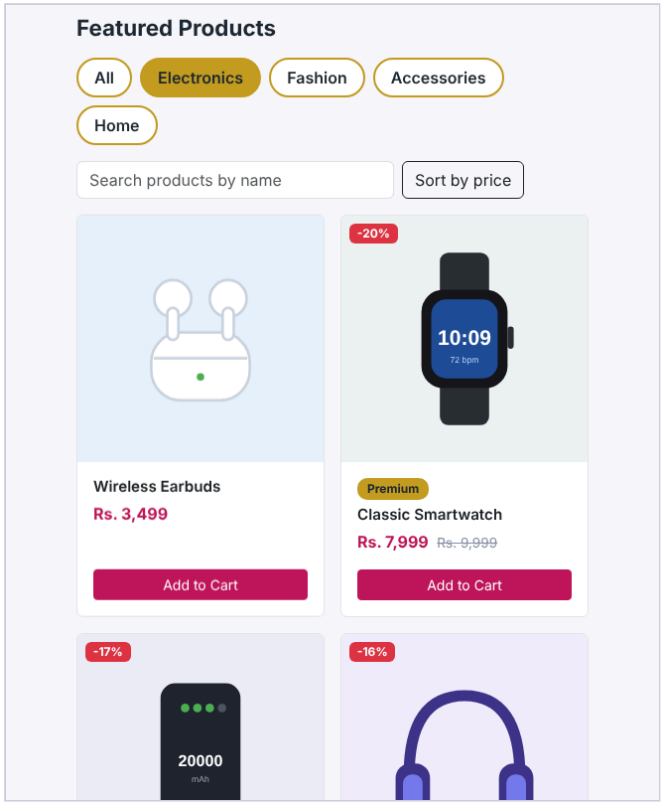
N	BP	H (accent)	K	Sticky top	Premium above
58	658px	46 → hsl(46, 70%, 45%)	4	88px	Rs. 5,800

Screenshots

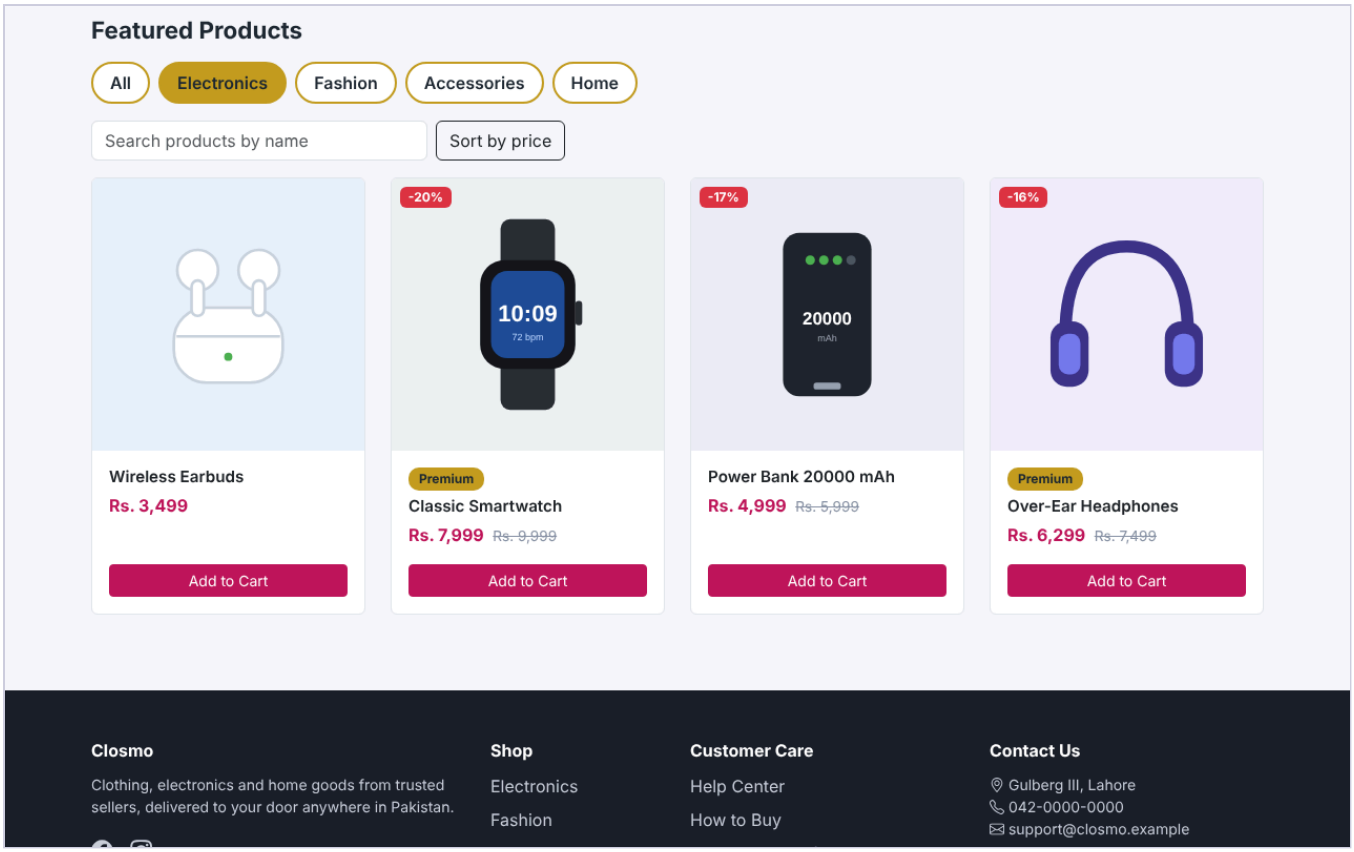
Task 1: CSS-only category filter ("Electronics" selected, so only the 4 electronics cards show, with no gaps)



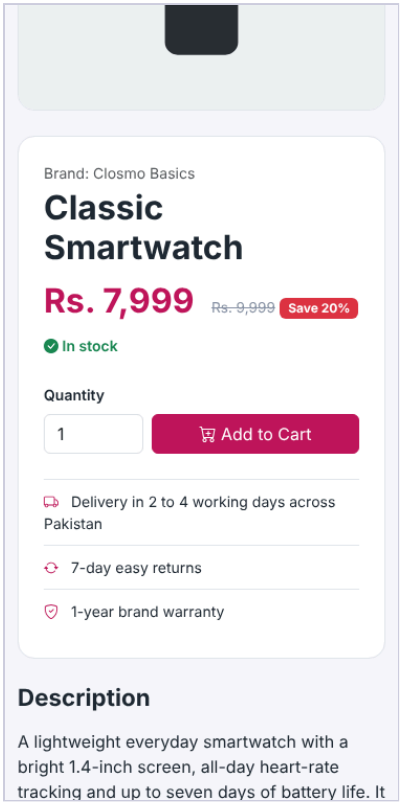
375px



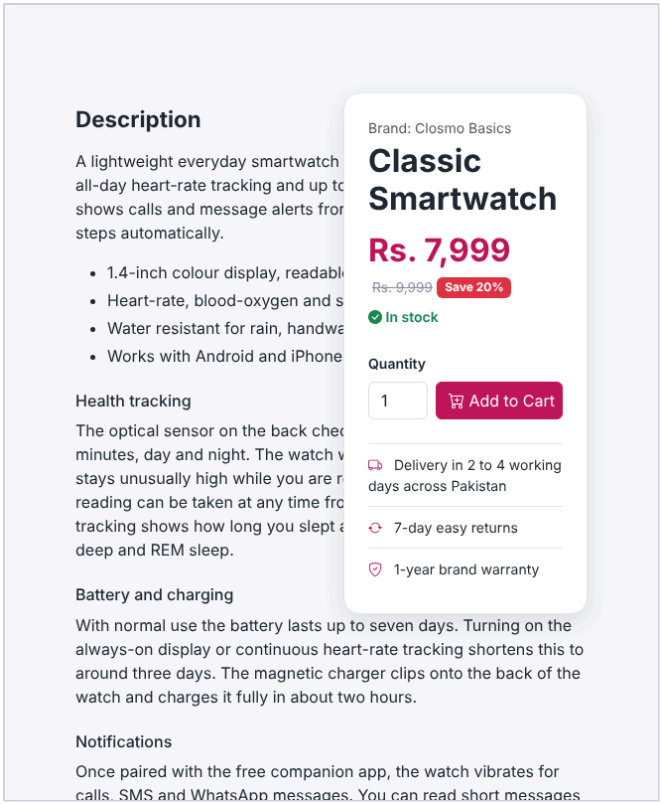
658px (BP)


Featured Products
All
Electronics
Fashion
Accessories
Home
Search products by name
Sort by price
-20%
10:09
72 bpm
-17%
20000
mAh
-16%
Wireless Earbuds
Rs. 3,499
Add to Cart
Premium
Classic Smartwatch
Rs. 7,999
Rs. 9,999
Add to Cart
Power Bank 20000 mAh
Rs. 4,999
Rs. 5,999
Add to Cart
Premium
Over-Ear Headphones
Rs. 6,299
Rs. 7,499
Add to Cart
Closmo
Clothing, electronics and home goods from trusted sellers, delivered to your door anywhere in Pakistan.
Shop
Electronics
Fashion
Customer Care
Help Center
How to Buy
Contact Us
Gulberg III, Lahore
042-0000-0000
support@closmo.example
1280px

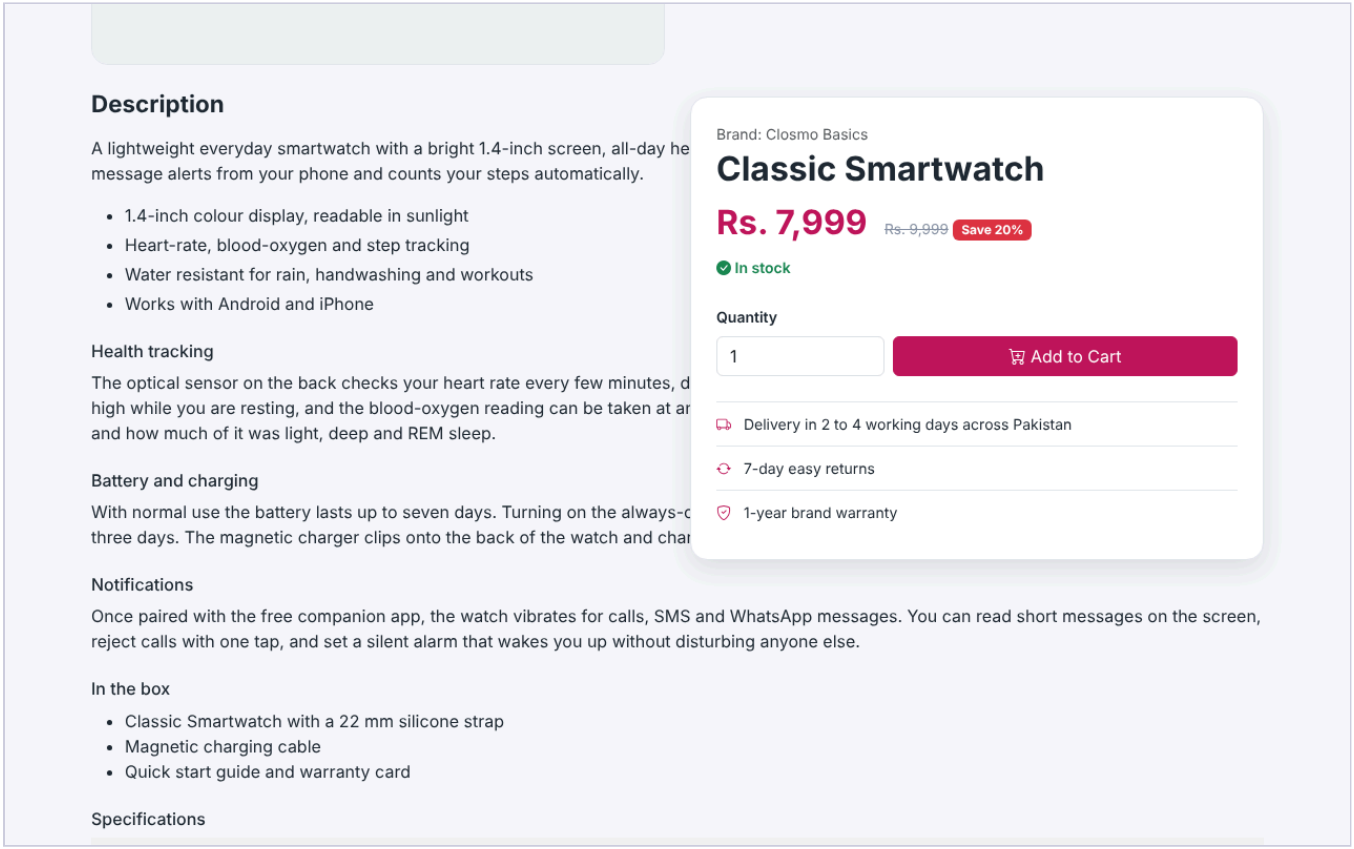
Task 2: product page with grid areas and a sticky details box



375px: image, then details, then description in one column (no sticky)

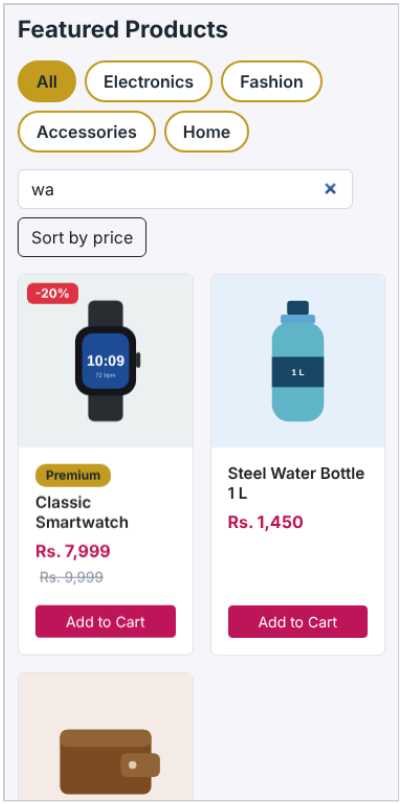


658px (BP): page scrolled 700px; details box stuck 88px from the top

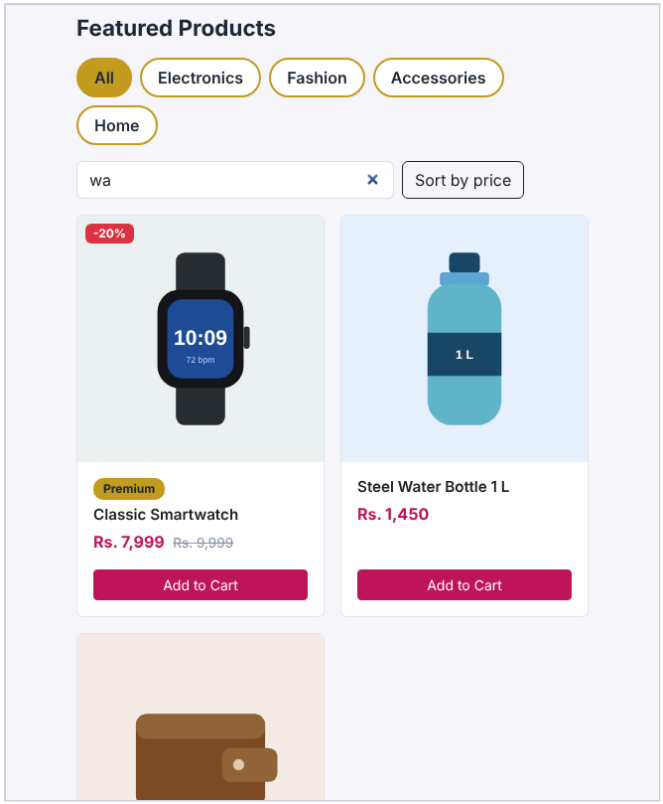


1280px: page scrolled 700px; image and details side by side, full-width description scrolling under the sticky details box

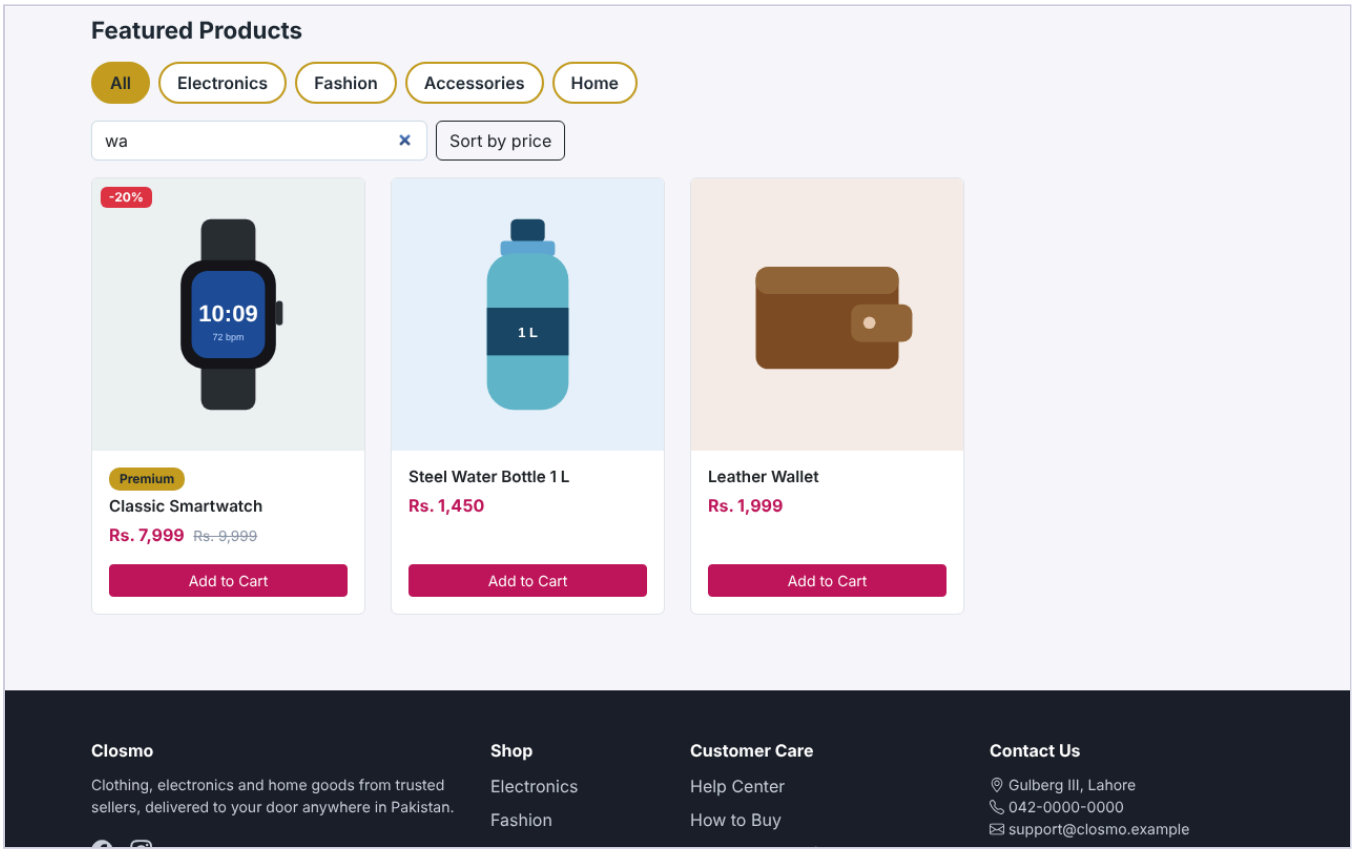
Task 3: live search (typed "wa": Classic Smartwatch, Steel Water Bottle, Leather Wallet; Premium label on the smartwatch)



375px

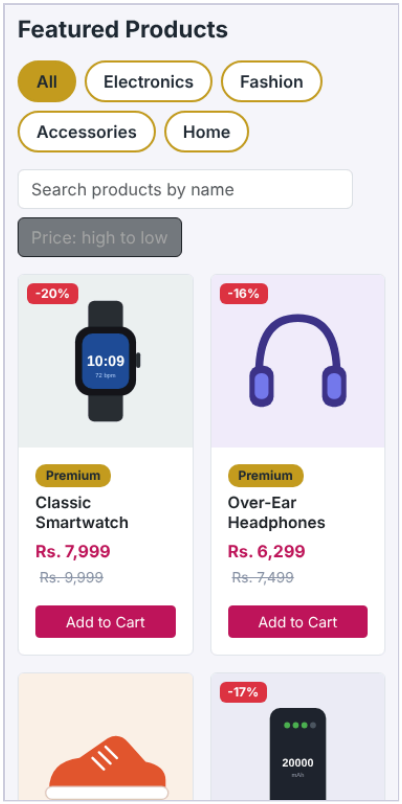


658px (BP)

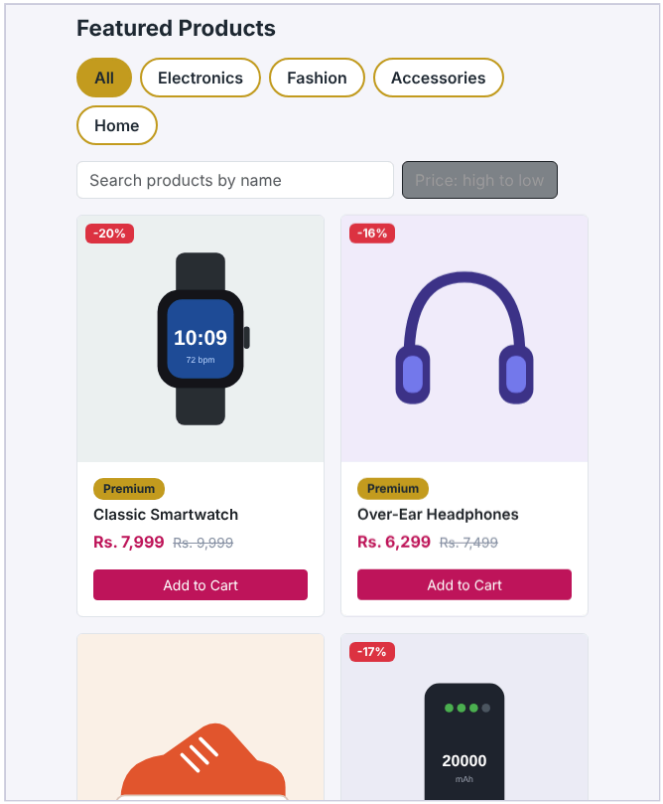


1280px

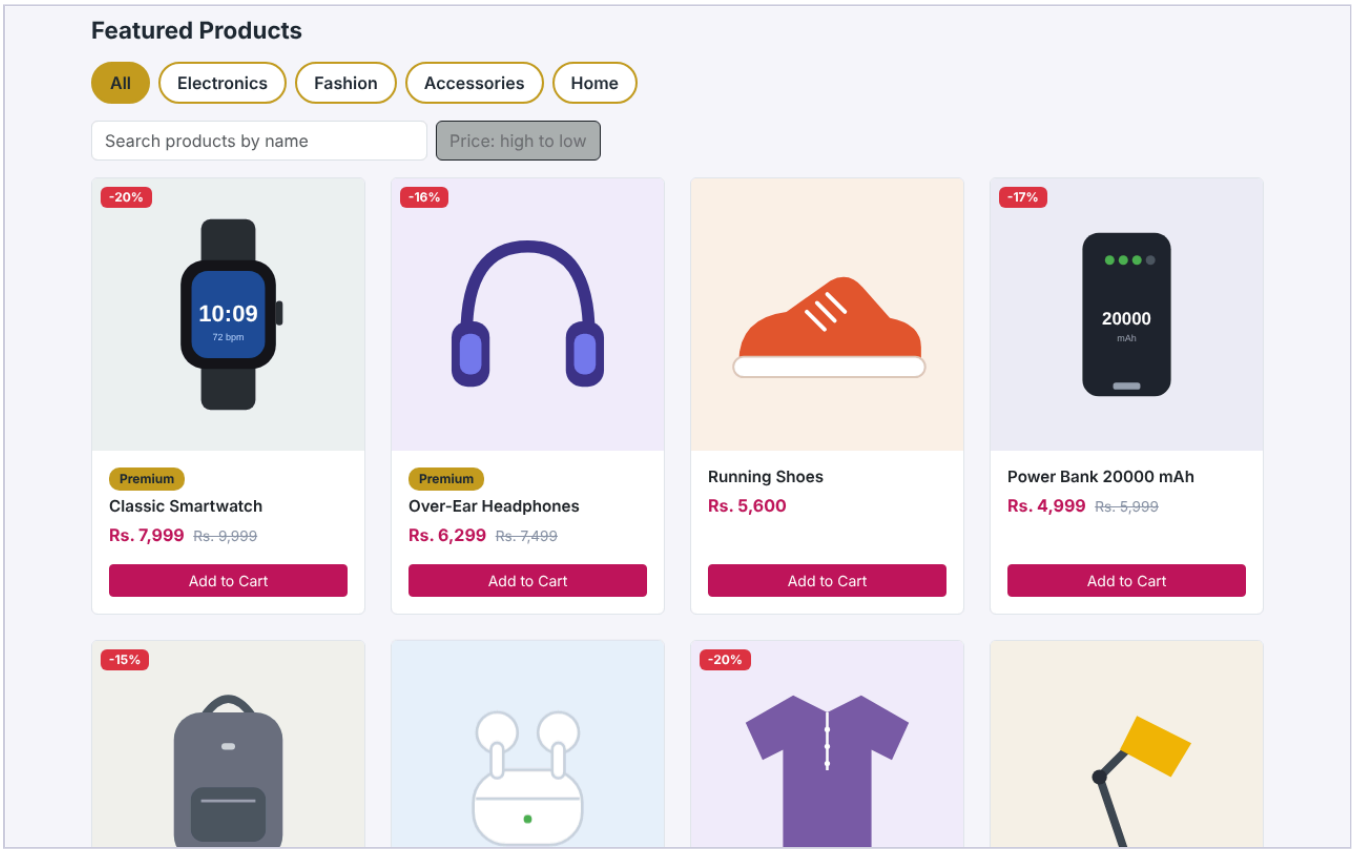
Task 3: Sort by price (clicked twice, so high to low; Premium labels on the Rs. 7,999 and Rs. 6,299 products)



375px

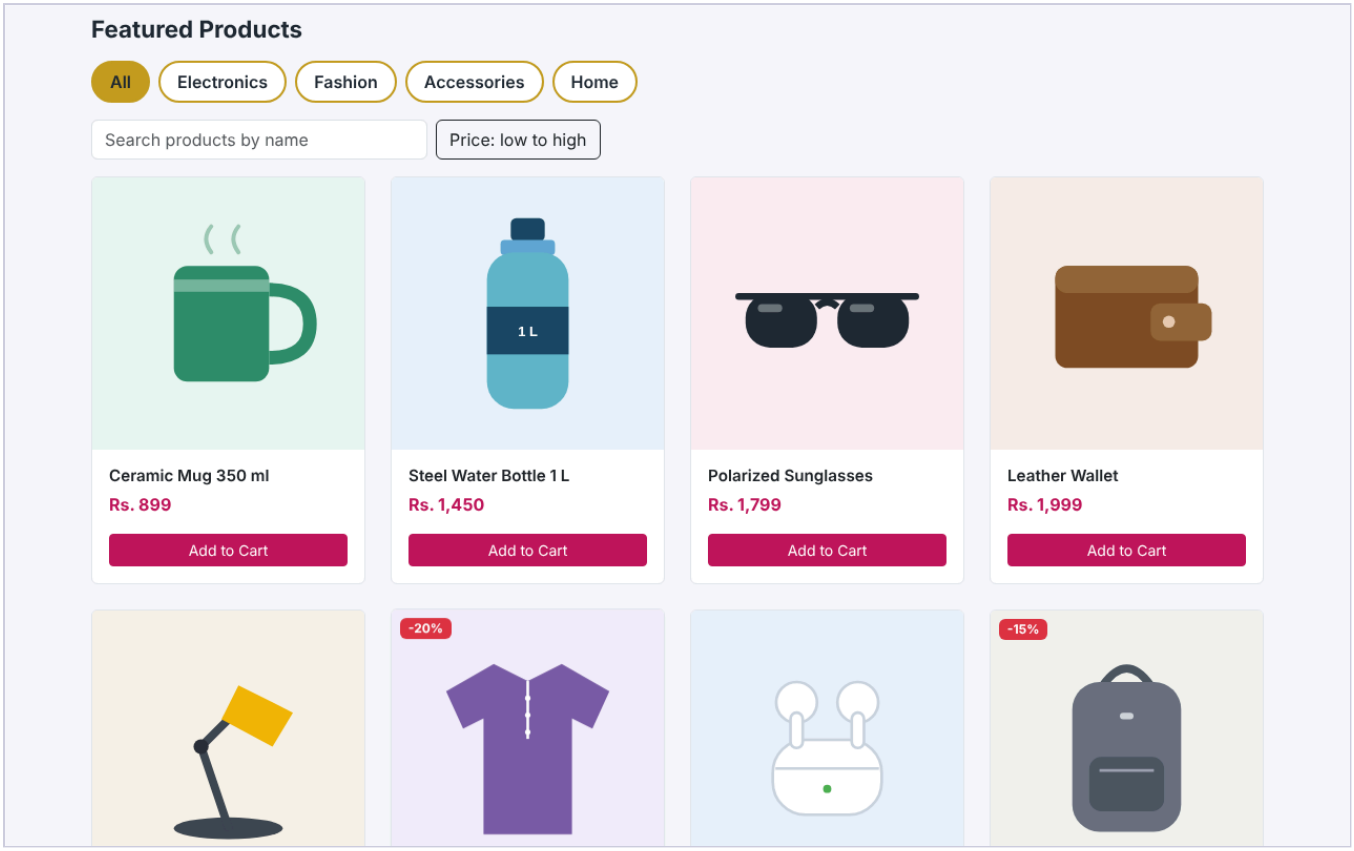


658px (BP)

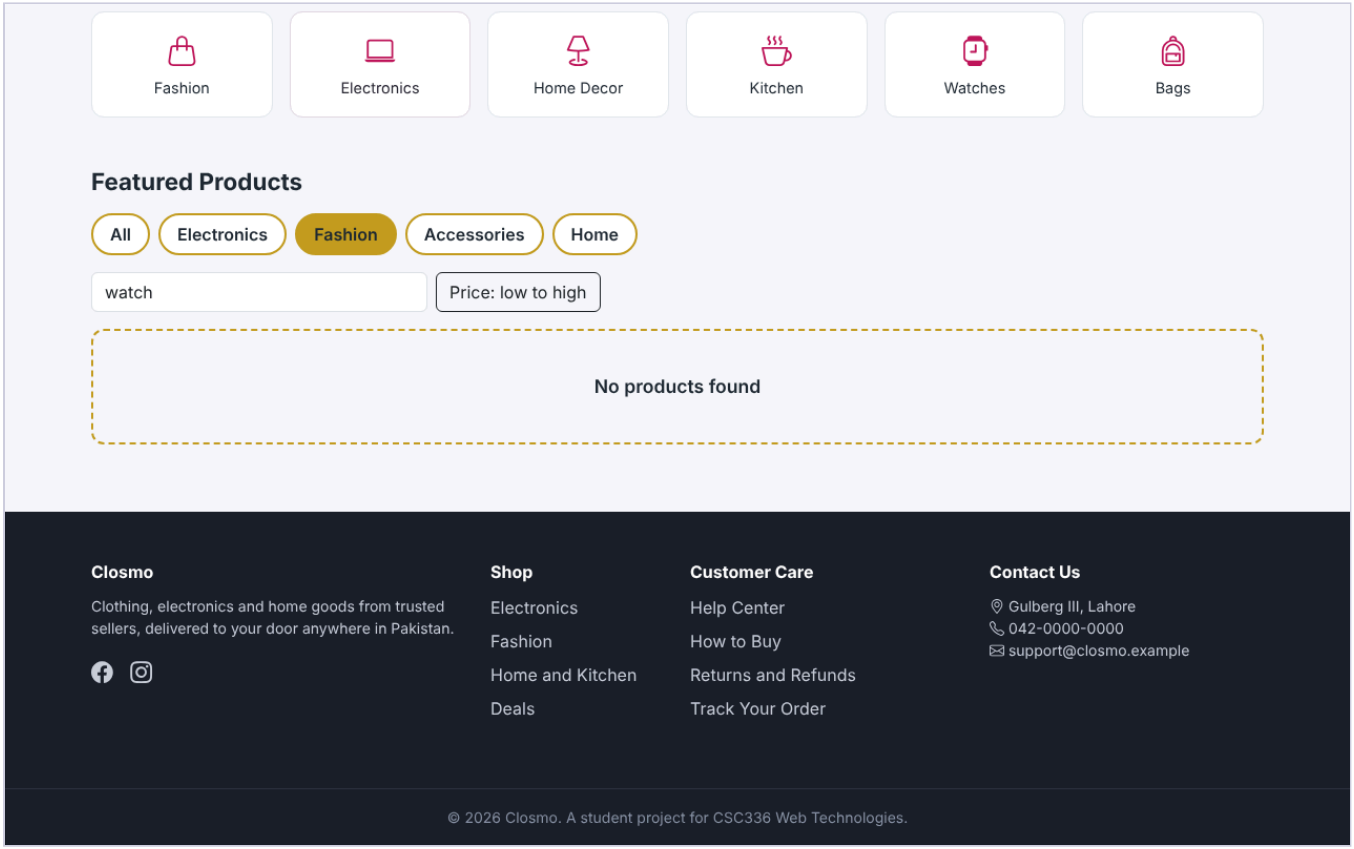


1280px

Task 3: Sort by price (first click: low to high), and search combined with the category filter



1280px, first click: Rs. 899 first



1280px: "watch" typed and Fashion selected; no card matches both, so "No products found" shows

Code: Task 1 (CSS-only category filter)

index.html: radio buttons added above the grid

```
<!-- MIDTERM TASK 1: CSS-only category filter (no JavaScript).
     K = 4 categories, so K + 1 = 5 radio buttons. "All" is checked on page load.
     Every radio has the same name="category", so only one can be selected.
     Each product column below has a class like cat-electronics, and the
     :has() rules in styles.css hide the columns that don't match. -->
<div class="filter-bar mb-3">
    <input type="radio" name="category" id="cat-all" checked>
    <label for="cat-all">All</label>

    <input type="radio" name="category" id="cat-electronics">
    <label for="cat-electronics">Electronics</label>

    <input type="radio" name="category" id="cat-fashion">
    <label for="cat-fashion">Fashion</label>

    <input type="radio" name="category" id="cat-accessories">
    <label for="cat-accessories">Accessories</label>

    <input type="radio" name="category" id="cat-home">
    <label for="cat-home">Home</label>
</div>
```

index.html: category class added to each of the 12 product columns

```
<div class="col-6 col-md-4 col-lg-3 product-item cat-fashion">
<div class="col-6 col-md-4 col-lg-3 product-item cat-electronics">
<div class="col-6 col-md-4 col-lg-3 product-item cat-electronics">
<div class="col-6 col-md-4 col-lg-3 product-item cat-fashion">
<div class="col-6 col-md-4 col-lg-3 product-item cat-accessories">
<div class="col-6 col-md-4 col-lg-3 product-item cat-fashion">
<div class="col-6 col-md-4 col-lg-3 product-item cat-home">
<div class="col-6 col-md-4 col-lg-3 product-item cat-electronics">
<div class="col-6 col-md-4 col-lg-3 product-item cat-home">
<div class="col-6 col-md-4 col-lg-3 product-item cat-home">
<div class="col-6 col-md-4 col-lg-3 product-item cat-electronics">
<div class="col-6 col-md-4 col-lg-3 product-item cat-accessories">
```

styles.css: added at the end

```
/* ============================================================
   MIDTERM - my numbers (roll FA24-BCS-058)
   N = 58, BP = 600 + 58 = 658px, H = (58 x 7) mod 360 = 46,
   K = (58 mod 3) + 3 = 4
   ============================================================ */
:root {
    --accent-h: hsl(46, 70%, 45%);   /* accent colour from my H */
}

/* ---------- TASK 1: CSS-only category filter ---------- */

/* The radio buttons sit in a row; flex-wrap lets them drop to a new line on phones */
.filter-bar {
    display: flex;
    flex-wrap: wrap;
    gap: 8px;
}

/* Hide the round radio dot. The <label for="..."> is what the user clicks,
   and clicking a label checks its radio. opacity: 0 (not display: none)
   keeps the radio usable with the keyboard. */
.filter-bar input {
    position: absolute;
    opacity: 0;
}

/* Each label looks like a pill-shaped button */
```

```
.filter-bar label {
    padding: 6px 16px;
    border: 2px solid var(--accent-h);
    border-radius: 20px;
    background-color: #ffffff;
    font-weight: 600;
    cursor: pointer;
}

/* The SELECTED filter: input:checked + label means
   "the label that comes straight after a checked radio" */
.filter-bar input:checked + label {
    background-color: var(--accent-h);
    color: var(--ink);
}

/* Keyboard users can see which filter has focus */
.filter-bar input:focus-visible + label {
    outline: 2px solid var(--ink);
    outline-offset: 2px;
}

/* THE FILTER ITSELF. Read the first line as:
   "if #products HAS a checked #cat-electronics radio, hide every
   .product-item that is NOT .cat-electronics".
   display: none removes the column completely, so Bootstrap's flex row
   moves the remaining cards up and no empty gaps are left. */
#products:has(#cat-electronics:checked) .product-item:not(.cat-electronics),
#products:has(#cat-fashion:checked) .product-item:not(.cat-fashion),
#products:has(#cat-accessories:checked) .product-item:not(.cat-accessories),
#products:has(#cat-home:checked) .product-item:not(.cat-home) {
    display: none;
}
```

Code: Task 2 (product page grid and sticky details)

product.html: the old Bootstrap row was replaced with this grid

```
<!-- MIDTERM TASK 2: one CSS Grid with three NAMED areas (see .product-layout in styles.css).
     Below 658px (my BP): image, then details, then description, in one column.
     From 658px: image and details side by side, description full width underneath,
     and the details box is sticky 88px from the top. -->
<div class="product-layout">

    <!-- grid-area: image -->
    <div class="product-image-area">
        <div class="product-image-box">
            <span class="badge bg-danger position-absolute top-0 start-0 m-3 fs-6">-20%</span>
            <img src="images/smartwatch.svg" class="img-fluid" alt="Black smartwatch with a blue
screen showing the time 10:09" width="600" height="600">
        </div>
    </div>

    <!-- grid-area: details (name, price, quantity, Add to Cart). This is the sticky box. -->
    <div class="product-details">
        <p class="text-muted small mb-1">Brand: Closmo Basics</p>
        <h1 class="product-title">Classic Smartwatch</h1>

        <p class="mb-2">
            <span class="price price-lg">Rs. 7,999</span>
            <del class="old-price">Rs. 9,999</del>
            <span class="badge text-bg-danger align-middle">Save 20%</span>
        </p>
        <p class="text-success fw-semibold small"><i class="bi bi-check-circle-fill" aria-
hidden="true"></i> In stock</p>

        <!-- Quantity selector + Add to Cart. Not functional yet.
             min/max are HTML5 validation attributes: the number box only allows 1 to 10. -->
        <div class="row g-2 align-items-end mt-3">
            <div class="col-4">
                <label for="quantity" class="form-label small fw-semibold">Quantity</label>
                <input type="number" class="form-control" id="quantity" name="quantity" min="1"
max="10" value="1">
            </div>
            <div class="col-8">
                <button type="button" class="btn btn-brand w-100 text-nowrap">
                    <i class="bi bi-cart-plus" aria-hidden="true"></i> Add to Cart
                </button>
            </div>
        </div>

        <ul class="list-unstyled delivery-info small mt-4 mb-0">
            <li><i class="bi bi-truck" aria-hidden="true"></i> Delivery in 2 to 4 working days across
Pakistan</li>
            <li><i class="bi bi-arrow-repeat" aria-hidden="true"></i> 7-day easy returns</li>
            <li><i class="bi bi-shield-check" aria-hidden="true"></i> 1-year brand warranty</li>
        </ul>
    </div>

    <!-- grid-area: description (long on purpose, so the page scrolls and the sticky box can be seen
working) -->
    <section class="product-description">
        <h2 class="section-title">Description</h2>
        <p>
            A lightweight everyday smartwatch with a bright 1.4-inch screen, all-day
            heart-rate tracking and up to seven days of battery life. It shows calls and
            message alerts from your phone and counts your steps automatically.
        </p>
        <ul class="feature-list">
            <li>1.4-inch colour display, readable in sunlight</li>
            <li>Heart-rate, blood-oxygen and step tracking</li>
            <li>Water resistant for rain, handwashing and workouts</li>
            <li>Works with Android and iPhone</li>
        </ul>

        <h3 class="h6 mt-4">Health tracking</h3>
        <p>
```

```
            The optical sensor on the back checks your heart rate every few minutes, day and night.
            The watch warns you if your heart rate stays unusually high while you are resting, and
            the blood-oxygen reading can be taken at any time from the health screen. Sleep tracking
            shows how long you slept and how much of it was light, deep and REM sleep.
        </p>

        <h3 class="h6 mt-4">Battery and charging</h3>
        <p>
            With normal use the battery lasts up to seven days. Turning on the always-on display
            or continuous heart-rate tracking shortens this to around three days. The magnetic
            charger clips onto the back of the watch and charges it fully in about two hours.
        </p>

        <h3 class="h6 mt-4">Notifications</h3>
        <p>
            Once paired with the free companion app, the watch vibrates for calls, SMS and
            WhatsApp messages. You can read short messages on the screen, reject calls with one
            tap, and set a silent alarm that wakes you up without disturbing anyone else.
        </p>

        <h3 class="h6 mt-4">In the box</h3>
        <ul>
            <li>Classic Smartwatch with a 22 mm silicone strap</li>
            <li>Magnetic charging cable</li>
            <li>Quick start guide and warranty card</li>
        </ul>

        <!-- Specifications are genuinely tabular data, so a table is the right element -->
        <h3 class="h6 mt-4">Specifications</h3>
        <div class="table-responsive">
            <table class="table table-striped spec-table">
                <tbody>
                    <tr><th scope="row">Display</th><td>1.4-inch colour touchscreen</td></tr>
                    <tr><th scope="row">Battery life</th><td>Up to 7 days</td></tr>
                    <tr><th scope="row">Sensors</th><td>Heart rate, blood oxygen, accelerometer</td>
</tr>
                    <tr><th scope="row">Connectivity</th><td>Bluetooth 5.3</td></tr>
                    <tr><th scope="row">Compatibility</th><td>Android and iOS</td></tr>
                    <tr><th scope="row">Strap</th><td>Silicone, 22 mm</td></tr>
                    <tr><th scope="row">Warranty</th><td>1 year</td></tr>
                </tbody>
            </table>
        </div>
    </section>

</div>
```

styles.css: added

```
/* ---------- TASK 2: product page grid with named areas + sticky details ---------- */

/* Phones (below 658px): ONE column. Each string in grid-template-areas is one
   row, so the order is image, then details, then description. */
.product-layout {
    display: grid;
    gap: 24px;
    grid-template-areas:
        "image"
        "details"
        "description";
}

/* Put each box into its named area */
.product-image-area  { grid-area: image; }
.product-details     { grid-area: details; }
.product-description { grid-area: description; }

/* The details box looks like a card */
.product-details {
    padding: 24px;
    background-color: #ffffff;
    border: 1px solid #e3e6ee;
    border-radius: 16px;
}
```

```
/* From my BP (658px) and wider: TWO columns.
   Row 1 = image and details side by side, row 2 = description across both columns. */
@media (min-width: 658px) {
    .product-layout {
        grid-template-columns: 1fr 1fr;
        grid-template-areas:
            "image       details"
            "description description";
        align-items: start;          /* boxes keep their own height instead of stretching */
    }

    .product-details {
        position: sticky;            /* normal position, until scrolling would push it off screen */
        top: 88px;                   /* (58 mod 40) + 70 = 88px from the top of the viewport */
        z-index: 1;                  /* stays above the description text while it scrolls past */
        box-shadow: 0 8px 24px rgba(0, 0, 0, 0.10);
    }
}
```

Code: Task 3 (live search, price sort, Premium label)

index.html: search box, sort button and message added above the grid; id added to the grid

```
<!-- MIDTERM TASK 3: search box and sort button. They only work because of script.js. -->
<div class="d-flex flex-wrap gap-2 mb-3">
    <input type="search" id="search-box" class="form-control search-box" placeholder="Search products
by name" aria-label="Search products by name">
    <button type="button" id="sort-btn" class="btn btn-outline-dark">Sort by price</button>
</div>

<!-- Shown by script.js only when no card matches -->
<p id="no-results" class="no-results" hidden>No products found</p>

<div class="row g-3 g-md-4" id="product-grid">

<!-- MIDTERM TASK 3: my JavaScript. Loaded at the end of <body>, so all the cards already exist when
it runs. -->
<script src="script.js"></script>
```

script.js: new file

```
// ========================================================
// MIDTERM TASK 3 - live search, price sort and Premium labels
// Plain JavaScript. It works with the product cards that are
// already in index.html; it never rebuilds them.
// ========================================================

const N = 58;                                   // last two digits of FA24-BCS-058

// Grab the elements we need from the page (the DOM)
const grid = document.getElementById('product-grid');
const searchBox = document.getElementById('search-box');
const sortBtn = document.getElementById('sort-btn');
const noResults = document.getElementById('no-results');
const categoryRadios = document.querySelectorAll('input[name="category"]');
const items = document.querySelectorAll('.product-item');   // the 12 product columns

// Reads the price from a card: "Rs. 3,499" -> 3499
// replace(/[^0-9]/g, '') deletes every character that is not a digit.
function getPrice(item) {
    const text = item.querySelector('.price').textContent;
    return Number(text.replace(/[^0-9]/g, ''));
}

// ---------- 1. "Premium" label, added once when the page loads ----------
// N x 100 = 5800, so any product above Rs. 5,800 gets the label.
items.forEach(function (item) {
    if (getPrice(item) > N * 100) {
        const label = document.createElement('span');   // make a new <span>
        label.className = 'premium-label';
        label.textContent = 'Premium';
        item.querySelector('.card-body').prepend(label); // put it at the top of the card body
    }
});

// ---------- 2. Live search (works together with the Task 1 category filter) ----------
function applySearch() {
    const typed = searchBox.value.trim().toLowerCase();
    let visibleCount = 0;

    items.forEach(function (item) {
        const name = item.querySelector('.product-name').textContent.toLowerCase();

        // hidden = true hides the card. Both sides are lowercase, so the search is not case-
sensitive.
        item.hidden = !name.includes(typed);

        // A card is really visible only if the search did NOT hide it AND the CSS
        // category filter did not hide it. getComputedStyle tells us the final display value.
        if (getComputedStyle(item).display !== 'none') {
            visibleCount++;
```

```
        }
    });

    // Show "No products found" only when nothing is visible
    noResults.hidden = visibleCount > 0;
}

// 'input' fires on every key press, so the results update while the user types
searchBox.addEventListener('input', applySearch);

// When the category changes, check again, so "No products found" stays correct
categoryRadios.forEach(function (radio) {
    radio.addEventListener('change', applySearch);
});

// ---------- 3. Sort by price (low -> high, then high -> low, and so on) ----------
let lowToHigh = true;

sortBtn.addEventListener('click', function () {
    const cards = Array.from(grid.children);   // turn the grid's children into a real array

    cards.sort(function (a, b) {
        // negative result = a goes first, positive = b goes first
        return lowToHigh ? getPrice(a) - getPrice(b) : getPrice(b) - getPrice(a);
    });

    // appendChild MOVES an element that is already on the page to the end of the grid.
    // Appending them in sorted order puts the whole grid in that order (no innerHTML).
    cards.forEach(function (card) {
        grid.appendChild(card);
    });

    sortBtn.textContent = lowToHigh ? 'Price: low to high' : 'Price: high to low';
    lowToHigh = !lowToHigh;                     // flip the direction for the next click
});
```

styles.css: added

```
/* ---------- TASK 3: styles for the parts added by JavaScript ---------- */
.search-box {
    max-width: 320px;
}

/* "Premium" label that script.js adds to products above Rs. 5,800 */
.premium-label {
    align-self: flex-start;          /* card-body is a flex column; this stops the label stretching
*/
    margin-bottom: 6px;
    padding: 2px 10px;
    border-radius: 10px;
    background-color: var(--accent-h);
    color: var(--ink);
    font-size: 0.75rem;
    font-weight: 700;
}

/* "No products found" message */
.no-results {
    padding: 40px 16px;
    border: 2px dashed var(--accent-h);
    border-radius: 12px;
    text-align: center;
    font-size: 1.1rem;
    font-weight: 600;
}
```